Automatic Stand Mixer

by
LELIS, KATHRYNA

CPE161P-4_C1
REAL-TIME EMBEDDED SYSTEM

MAPÚA
UNIVERSITY

PROJECT OVERVIEW

The **Automatic Stand Mixer** is an Arduino-based kitchen appliance designed to automate and simplify mixing tasks through embedded systems and real-time monitoring. It features adjustable speed settings (50%, 75%, 100%) and timed mixing modes (30 seconds to 6 minutes), controlled via a keypad or IR remote. A servo motor automates beater movement, while a DC motor ensures precise mixing. Real-time feedback is provided through an LCD display, RGB LEDs (indicating speed levels), and a buzzer for alerts. Housed in a lightweight cardboard casing, the prototype combines functionality with an aesthetic design inspired by commercial mixers. Beyond its practical use, the project serves as an educational tool, demonstrating microcontroller-based automation and inspiring further innovation in kitchen technology.

MAPÚA UNIVERSITY

INTRODUCTION

Objective Overview:

- **To develop an Automated Smart Mixer System.**
- **To demonstrate the use of LEDs and a buzzer.**
- **To apply concepts from logic circuits, microcontrollers, and embedded systems.**
- **To implement a user-friendly interface using an LCD I2C display**
- **To integrate remote control functionality using an IR sensor and remote**
- **To utilize a servo motor.**

MAPÚA UNIVERSITY

PROJECT OVERVIEW

- **Components Used:**

Quantity	Material	Description
1	Keypad	For setting speed, time and on/off
1	Buzzer	For mixing is complete and overload detected alert
1	RGB LED	Indicator of mixing speed as well as on/off
1	Servo	For automatically lowering and lifting the beater
1	LCD I2C	For User Interface
1	IR Remote & Sensor	For setting up speed, time, and on/off remotely
1	DC Motor	For mixing

MAPÚA UNIVERSITY

METHODOLOGY

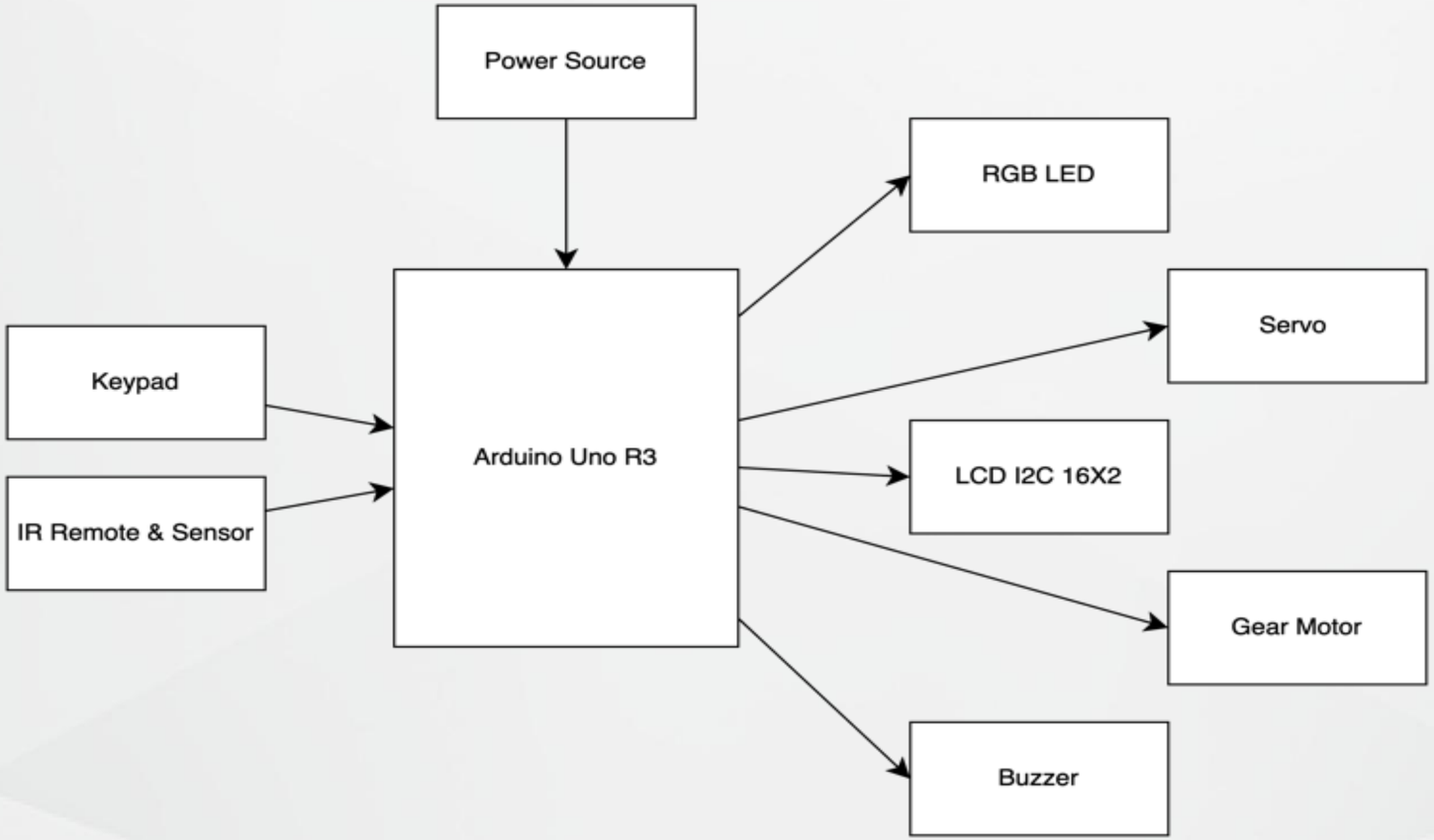
Device Mechanics

The Automatic Stand Mixer is designed to automate the process of mixing ingredients with adjustable speed settings and timed mixing modes. The primary objective is to provide users with a convenient, efficient, and hands-free mixing experience. Users can control the mixer using a keypad or IR remote, setting the desired speed (50%, 75%, or 100%) and mixing duration (30 seconds to 6 minutes). The system provides real-time feedback through an LCD display, RGB LEDs, and a buzzer, ensuring users are informed about the mixer's status at all times.

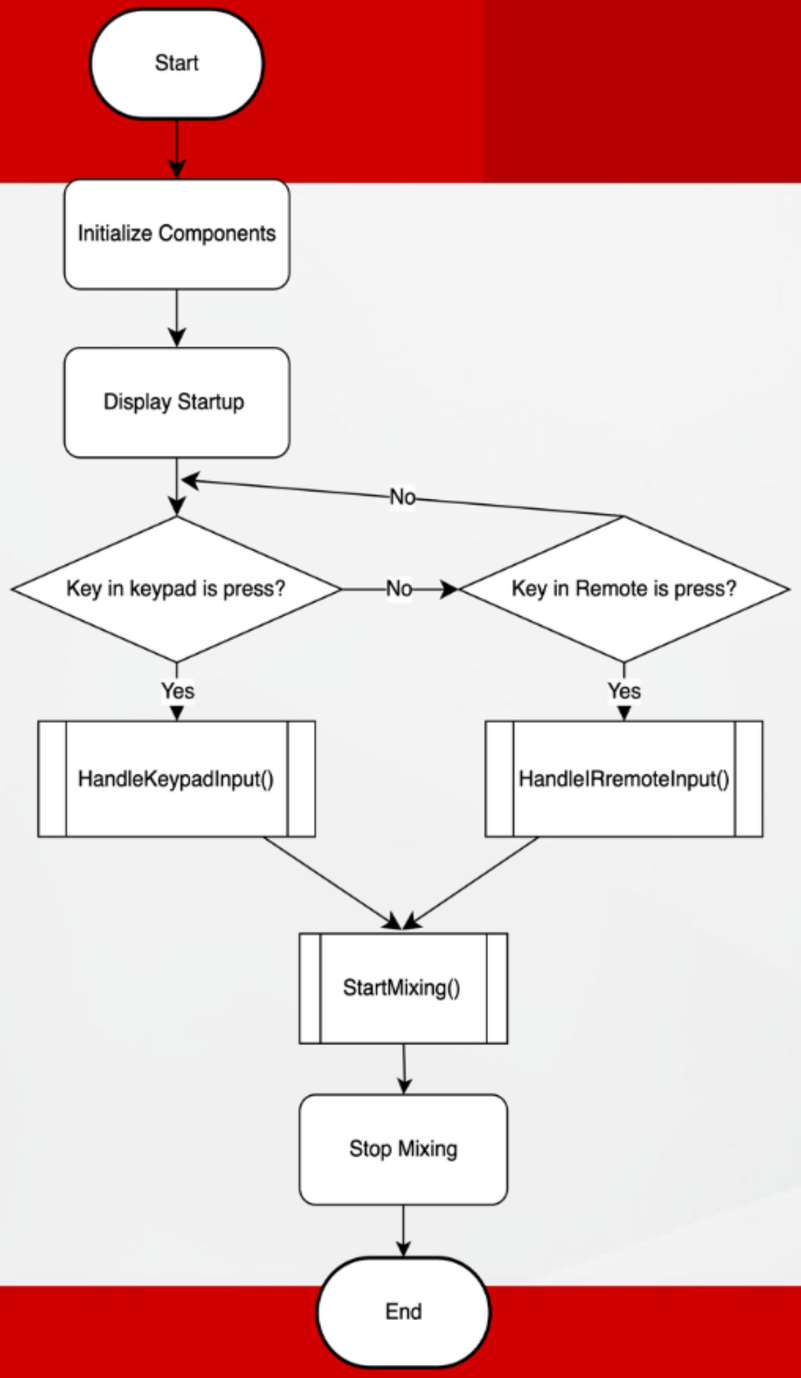
Device Flow

- Initialization: System powers on, LCD shows "Select Speed," and the beater resets to 15°.
- User Input: Keypad/IR sets speed or timed mixing.
- Mixing: Beater tilts to 40°, motor runs at set speed, LEDs and LCD update in real-time.
- Completion: Motor stops, beater resets, buzzer sounds, and LCD displays "MIXING COMPLETE."
- Reset: System returns to idle, ready for the next operation.

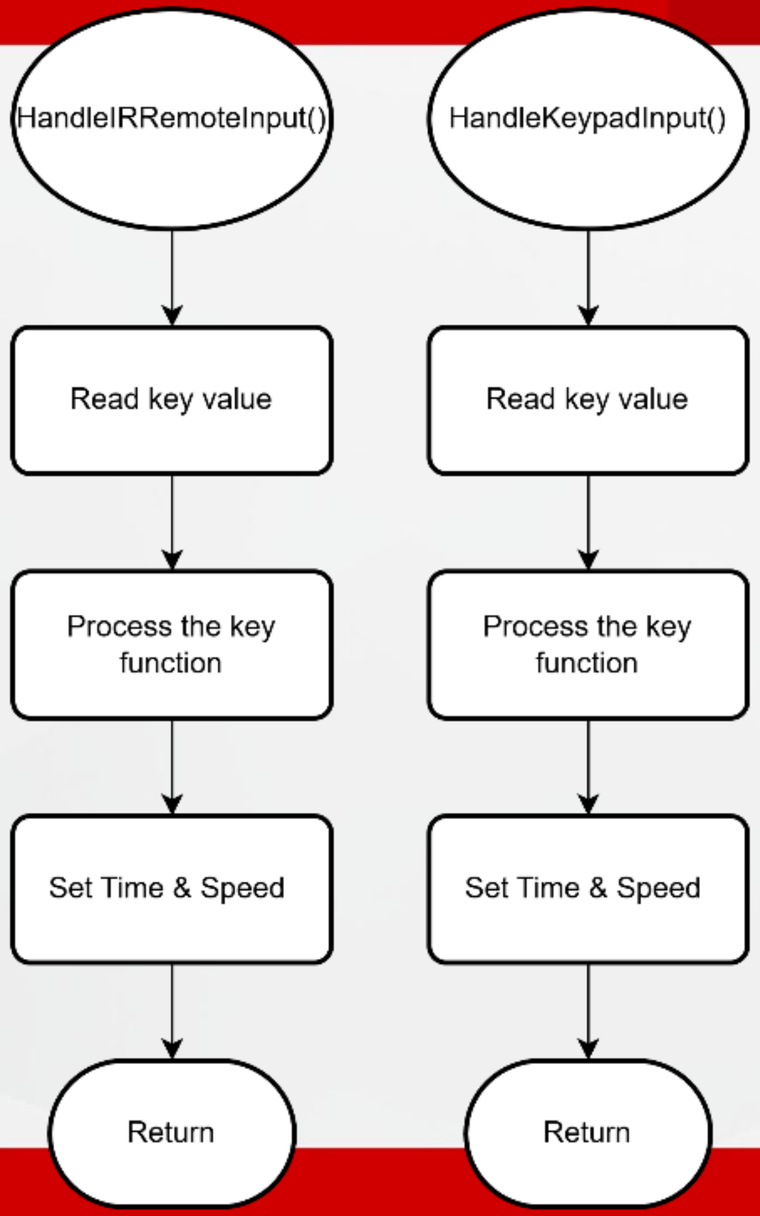
HARDWARE BLOCK DIAGRAM



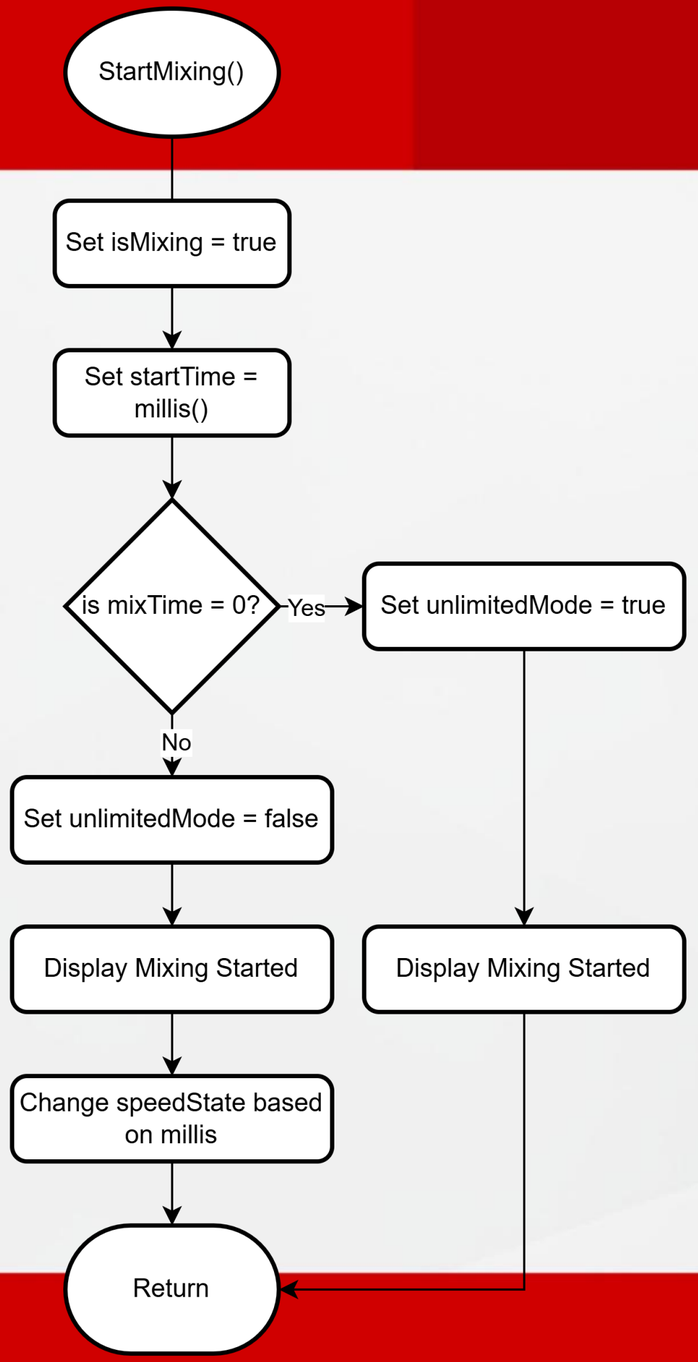
SOFTWARE FLOWCHART



SOFTWARE FLOWCHART



SOFTWARE FLOWCHART



Program Listing with Comment

```
#include <Servo.h>
#include <IRremote.hpp>
#include <Keypad.h>
#include <LiquidCrystal_I2C.h>

// Pin Definitions
const int ENA = 6;          // Enable pin for motor speed control (PWM)
const int IN1 = 8;          // Motor direction control pin
const int LED_LOW = 7;      // Pin for green LED (50% speed)
const int LED_MEDIUM = 9;   // Pin for blue LED (75% speed)
const int LED_HIGH = 5;     // Pin for red LED (100% speed)
const int BUZZER = 3;       // Pin for buzzer
const int SERVO_PIN = 10;   // Pin for servo motor

// LCD I2C Setup (16x2 display)
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Keypad Setup
const byte ROWS = 4;        // Number of rows in the keypad
const byte COLS = 3;        // Number of columns in the keypad
```

```
char hexaKeys[ROWS][COLS] = {
  {'1', '2', '3'},
  {'4', '5', '6'},
  {'7', '8', '9'},
  {'*', '0', '#'}
};
byte rowPins[ROWS] = {A3, A2, A1, A0}; // Pins connected to keypad rows
byte colPins[COLS] = {11, 12, 13};     // Pins connected to keypad columns
Keypad customKeypad = Keypad(makeKeymap(hexaKeys), rowPins, colPins, ROWS, COLS);

// IR Remote Codes (Raw data for each button)
#define UNLI_50 0xBA45FF00   // Code for 50% speed (unlimited mode)
#define UNLI_75 0xB946FF00   // Code for 75% speed (unlimited mode)
#define UNLI_100 0xB847FF00  // Code for 100% speed (unlimited mode)
#define STOP 0xBC43FF00      // Code to stop the mixer
#define TIMER_0 0xE916FF00   // Code for 30-second timer
#define TIMER_1 0xF30CFF00   // Code for 1-minute timer
#define TIMER_2 0xE718FF00   // Code for 2-minute timer
#define TIMER_3 0xA15EFF00   // Code for 3-minute timer
#define TIMER_4 0xF708FF00   // Code for 4-minute timer
```

MAPÚA UNIVERSITY

Program Listing with Comment

```
#define TIMER_5 0xE31CFF00   // Code for 5-minute timer
#define TIMER_6 0xA55AFF00   // Code for 6-minute timer

// Mixing Parameters
const unsigned long MIX_TIMES[] = {30000, 60000, 120000, 180000, 240000, 300000, 360000}; // Mixing times (30s to 6min)
const int SPEED_STATES[] = {50, 75, 100}; // Speed levels (50%, 75%, 100%)
const int PWM_VALUES[] = {127, 192, 255}; // PWM values corresponding to speed levels

// Global Variables
Servo tiltServo;             // Servo motor object
int speedState = 50;         // Current speed state (default: 50%)
bool isMixing = false;       // Flag to check if mixer is active
unsigned long startTime = 0; // Start time for mixing
unsigned long mixTime = 0;   // Total mixing time
bool unlimitedMode = false;  // Flag for unlimited mixing mode
```

Program Listing with Comment

```
void setup() {
  Serial.begin(9600);        // Initialize serial communication
  IrReceiver.begin(2, ENABLE_LED_FEEDBACK); // Initialize IR receiver

  // Set pin modes
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(LED_LOW, OUTPUT);
  pinMode(LED_MEDIUM, OUTPUT);
  pinMode(LED_HIGH, OUTPUT);
  pinMode(BUZZER, OUTPUT);

  tiltServo.attach(SERVO_PIN); // Attach servo motor
  tiltServo.write(15);         // Set servo to idle position (15°)

  lcd.init();                  // Initialize LCD
  lcd.backlight();             // Turn on LCD backlight
  resetToIdle();               // Reset system to idle state
}

void loop() {
  handleKeypadInput();         // Check for keypad input
  handleIRInput();             // Check for IR remote input
}
```

Program Listing with Comment

```
// Function to handle keypad input
void handleKeypadInput() {
  char customKey = customKeypad.getKey(); // Read keypad input
  if (customKey != NO_KEY) {
    switch (customKey) {
      case '0': case '1': case '2': case '3': case '4': case '5': case '6':
        startTimedMixing(MIX_TIMES[customKey - '0']); // Start timed mixing based on key
        break;
      case '7': setSpeed(50); break; // Set speed to 50%
      case '8': setSpeed(75); break; // Set speed to 75%
      case '9': setSpeed(100); break; // Set speed to 100%
      case '*': stopMixing(); break; // Stop mixing
    }
  }
}
```

```
// Function to handle IR remote input
void handleIRInput() {
  if (IrReceiver.decode()) {
    uint32_t decodedValue = IrReceiver.decodedIRData.decodedRawData; // Decode IR signal
    switch (decodedValue) {
      case UNLI_50: setSpeed(50); break; // Set speed to 50%
      case UNLI_75: setSpeed(75); break; // Set speed to 75%
      case UNLI_100: setSpeed(100); break; // Set speed to 100%
      case STOP: stopMixing(); break; // Stop mixing
      case TIMER_0: startTimedMixing(MIX_TIMES[0]); break; // Start 30-second timer
      case TIMER_1: startTimedMixing(MIX_TIMES[1]); break; // Start 1-minute timer
      case TIMER_2: startTimedMixing(MIX_TIMES[2]); break; // Start 2-minute timer
      case TIMER_3: startTimedMixing(MIX_TIMES[3]); break; // Start 3-minute timer
      case TIMER_4: startTimedMixing(MIX_TIMES[4]); break; // Start 4-minute timer
      case TIMER_5: startTimedMixing(MIX_TIMES[5]); break; // Start 5-minute timer
      case TIMER_6: startTimedMixing(MIX_TIMES[6]); break; // Start 6-minute timer
    }
    IrReceiver.resume(); // Resume IR receiver
  }
}
```

MAPÚA UNIVERSITY

Program Listing with Comment

```
// Function to start timed mixing
void startTimedMixing(unsigned long duration) {
  startTime = millis(); // Record start time
  mixTime = duration;   // Set mixing duration
  isMixing = true;      // Set mixing flag
  unlimitedMode = false; // Disable unlimited mode

  while (millis() - startTime < mixTime) {
    unsigned long elapsedTime = millis() - startTime; // Calculate elapsed time

    // Adjust speed based on elapsed time
    if (elapsedTime < mixTime / 3) {
      setSpeed(100); // First phase: 100% speed
    } else if (elapsedTime < 2 * mixTime / 3) {
      setSpeed(75);  // Second phase: 75% speed
    } else {
      setSpeed(50);  // Final phase: 50% speed
    }

    tiltServo.write(40); // Tilt beater to 40°
    updateLCD();         // Update LCD display
  }
  stopMixing(); // Stop mixing after duration
}

// Function to set motor speed
void setSpeed(int speed) {
  speedState = speed; // Update speed state
  int pwmValue = PWM_VALUES[(speedState / 25) - 2]; // Map speed to PWM value
  analogWrite(ENA, pwmValue); // Set motor speed
  digitalWrite(IN1, HIGH);    // Enable motor
  setColorBasedOnSpeed();     // Update LED color
  updateLCD();                // Update LCD display
}
```

Program Listing with Comment

```
// Function to set LED color based on speed
void setColorBasedOnSpeed() {
  switch (speedState) {
    case 50: setColor(0, 255, 0); break; // Green for 50%
    case 75: setColor(255, 255, 0); break; // Yellow for 75%
    case 100: setColor(255, 0, 0); break; // Red for 100%
    default: setColor(0, 0, 0); break; // Off for other speeds
  }
}

// Function to set RGB LED color
void setColor(int red, int green, int blue) {
  analogWrite(LED_HIGH, red);   // Set red LED
  analogWrite(LED_LOW, green);  // Set green LED
  analogWrite(LED_MEDIUM, blue); // Set blue LED
}

// Function to stop mixing
void stopMixing() {
  analogWrite(ENA, 0); // Stop motor
  tiltServo.write(15); // Reset beater to 15°
  isMixing = false;    // Clear mixing flag
  tone(BUZZER, 1000, 500); // Play buzzer for 500ms
  delay(500);
  noTone(BUZZER);      // Stop buzzer
  setColor(0, 0, 0);   // Turn off LEDs
  resetToIdle();       // Reset system to idle state
}
```

```
// Function to reset system to idle state
void resetToIdle() {
  lcd.clear(); // Clear LCD
  lcd.setCursor(0, 0);
  lcd.print("Select Speed:"); // Display idle message
  lcd.setCursor(13, 0);
  lcd.print(speedState);
  lcd.print("%");
  lcd.setCursor(0, 1);
  lcd.print("Mixer Ready");
}

// Function to update LCD display
void updateLCD() {
  lcd.setCursor(0, 0);
  lcd.print("MIXING TIMER MODE"); // Display mixing mode
  lcd.setCursor(0, 1);
  lcd.print("SPEED: ");
  lcd.print(speedState); // Display current speed
  lcd.print("%  ");
}
```

MAPÚA UNIVERSITY

Prototype Demonstration

CPE160P-4_A1
EMBEDDED SYSTEM AND DESIGN

MAPÚA UNIVERSITY

THANK YOU FOR LISTENING!

MAPÚA UNIVERSITY